

# 软件质量测试

---

## 第1章 软件测试核心概念

软件

软件测试

软件缺陷

测试用例

自动化测试

## 第2章 软件测试背景

## 第3章 黑盒测试技术

边界值测试

等价类测试

基于决策表的测试

基于正交表的测试

基于场景的测试——基于用户行为的测试方法

## 第4章 黑盒测试案例实践

## 第5章 白盒测试技术

对判定的测试

对路径的测试

对循环的测试

静态白盒测试

对变量的测试

## 第6章 白盒测试案例实践

## 第7章 单元测试

## 第8章 集成测试

## 第9章 系统测试

# 第1章 软件测试核心概念

软件

软件 = 程序 + 数据库 + 文档 + 服务

软件特点：必须依托具体的硬件设备，依靠人的智力劳动，软件不会磨损

## 软件测试

🌱 软件测试：（IEEE1983）是使用**人工**和**自动手段**来运行或测试某个系统的过程，目的在于**检验其是否满足规定的需要**或是弄清楚预期结果与实际结果之间的差别

1、软件测试的**根本目的**是**确保软件满足用户需求**

2、软件测试的目的是要衡量软件产品是否符合预期

准确理解：系统需求规格说明书（SRS）验证：测试用例（Test Case）

3、软件测试是一个**持续进行**的过程

**计划测试，设计测试，实施测试（最重要）**

**执行测试：**（手动或者测试脚本）**单元**测试和**集成**测试可以同时，之后**系统**测试

**评估测试：执行结果，测试过程**

4、测试既需要**动态**执行也需要**静态**检查

5、测试不仅需要**手工**执行还需要**自动**执行

认知误区：

1、如果我们有良好的设计和高水平的程序员，就不需要测试了

很难保证0缺陷

2、软件测试并不创造任何代码和产品，我们可以不需要

3、测试测试等于调试

测试是发现缺陷的过程，并不知道是否有缺陷，调试是修复缺陷的过程，前提知道缺陷存在

4、软件需求规格说明应详细地包含所有用户需求

用户需求是软件测试的唯一标准，SRS是为了方便而呈现的用户需求，不用全部涵盖

5、软件测试可以提高

只测试不修复

6、软件质量测试是没有技术含量的

## 软件缺陷

简答🌱软件缺陷的定义——Ron Patton

软件测试员认为软件难以理解、不易使用、运行速度缓慢，或者最终用户认为不好

软件**未**达到需求规格说明书中指定的功能

软件**出现**了需求规格说明书中指明不会出现的错误（容错能力）

软件功能**超出**需求规格说明书中指定的范围

软件未达到需求规格说明书中虽未指出但应达到的目标

简答🌱软件测试员的主要任务

根据用户的**意见**和**反馈**执行测试

依据SRS，针对系统在**有效输入**及**有效操作**下的正常功能进行测试

依据SRS或个人经验，针对系统在**无效输入**或**无效操作**下的软件**容错能力**进行测试

开发人员应遵循**良好的开发习惯**，与用户和项目组成员**及时沟通**，避免植入无依据的软件缺陷

需求分析阶段强调**测试专家**的介入，从测试的视角完善需求规格说明，提高系统的外部环境容错能力

测试分析

测试改进：

针对需求进行**明确和细化**，在每个测试中有明确的**操作步骤和输入数据**，测试是可以对应实际进行执行的

对照系统的需求规格说明来设计测试，至少可以满足测试是对应**功能点进行覆盖**的

进一步改善：测试的完整性和有效性、代码的测试、测试的管理

## 测试用例

简答🌴测试用例的定义——IEEE1990

是一组**测试输入、执行条件和预期结果**，目的是要**满足一个特定的目标**，比如**执行一条特定的程序路径或检验是否符合一个特定的需求**

**测试用例 = 输入（输入数据和操作步骤） + 输出 + 测试环境**

输入数据：正常数据、错误数据、边界数据

## 自动化测试

定义：它是通过**测试工具、测试脚本**(Test Scripts)等手段，按照测试工程师的预定计划对软件产品进行自动的测试，从而验证软件是否满足用户的需求。

可重复，可操作，高效率

通过工具是无法完成自动生成测试用例的

技术：**录制/回放技术和脚本技术**（线性脚本、结构化脚本、共享脚本、数据驱动脚本、关键字驱动脚本）

# 第2章 软件测试背景

软件的测试的发展历程

第一阶段：初始阶段：直到**1957**年，软件测试才开始和调试区分开来

第二阶段：定义阶段：里程碑——1、1972年举行历史上第一次以软件测试为主题的正式学术会议2、1979年，软件测试领域第一本最重要的专著

第三阶段：集成阶段

第四阶段：管理、测量和最佳化阶段

外包测试现状

三种模式

1、现场测试模式(On-Site)

2、内部测试模式(In-House)

完全离岸外包模式(Off Shore)

现场增援与离岸结合模式(On Site+Off Shore)

3、设立联合研发中心模式

总体职业特点：人才需求大、职业稳定、无性别歧视、但仍需掌握一定的业务和技术能力

## 第3章 黑盒测试技术

5种测试方法：**边界值测试**、**等价类测试**、**基于决策表的测试**和**基于正交表的测试**（**单元测试**阶段，用于对**函数或类**的方法进行测试）**基于场景的测试**（是从**业务流程**优选的角度展开测试，系统测试，用于对功能、界面等进行测试）

黑盒测试仅需要知道对象的输入和预期输出

优势：1、对测试人员的技术要求相对较低**2**、不需要了解程序实现的**细节**，测试团队与开发团队可以并行完成各自的任务

局限：覆盖度不容易度量，测试的潜在风险较高——>通过白盒测试评估黑盒的测试覆盖率

被测对象

函数：完成对函数功能的测试，无需看函数代码，只需了解函数**接口和返回值**（**接口测试**）对应单元测试阶段

功能：完成对整个软件系统**功能和易用性**等的测试，无需看各功能点如何编程实现，只需要了解**SRS中关于输入和输出的规定对应系统测试**，或有用户共同参与的验收测试阶段

### 边界值测试

基本原理：在被测对象的**边界及边界附近设计测试用例**

**选择被测对象**：即输入域或输出域，以进行后续的边界值测试用例设计

**确定边界**：即输入/输出域的边界，确保覆盖被测对象所有可能的边界

**确定邻域**：即输入/输出域边界附近的邻域范围，便于及时发现所有潜伏在边界附近的缺陷

**设计用例**：即从边界及其邻域抽取测试数据，设计测试用例

输入域的确定：整体输入域（多个输入条件、边界清晰，难以覆盖），个体输入域（单个、边界不清晰）

边界的确定：取值范围，值的个数，有序集合

独立性假设：假设各个输入条件之间相互独立，不产生相互影响，即不具有相互依赖关系

领域：每个边界点(设为P点)，需在该点附近确定大小为1的邻域

测试用例：

1、测试数据的选择：

穷举法：在每个边界点的邻域范围内取所有数据——>所有数据均可测试，负担重

典型值法：a处选择a-1,a,a+1

## 2、边界组合方式

**强边界法**：覆盖所有输入条件的所有边界组合，每个测试用例对应多个输入条件同时取边界测试数据

**弱边界法**：单缺陷假设，优势在于便于快速隔离和定位边界缺陷，且大大降低测试用例

全边界法：强边界+弱边界

## 3、测试方案

穷举法+全边界、法穷举法+强边界法、典型值法+强边界法、**典型值法+弱边界法（常用）**

小结：边界值测试是一种**最基本，最简单**的黑盒测试方法，通常可作为等价类测试的补充

基于：**独立性假设+单缺陷假设**，关注的是系统边界，并不关注系统对不同类型数据的处理规律

## 等价类测试

测试方法目标：测试的**完备性**，测试的**无冗余性**

**基于：独立性假设+单缺陷假设**

原理：通过等价划分的方法来将**数据分片**

**将输入域划分为子集，称为等价类**

子集满足：每个子集内所有数据等价（覆盖），各个子集之间互不相交（无冗余），所有子集的并集是整个输入域（完备）

有效等价类：合理，有意义，被测对象能接受，考察软件的**正常工作能力**

无效等价类：不合理，无意义，被测对象不能接收，**容错能力**

划分原则：将某个输入条件所有可能的取值划分为一个有效，其余无效，再细化

等价类测试适用于：可用数量衡量的独立变量、布尔变量、逻辑变量

**等价类测试不适用于：相互有依赖关系的变量**

## 设计测试用例

有效等价类：强组合；无效等价类：单缺陷假设

分类的决定因素：<一般,健壮>，<单缺陷（弱），多缺陷（强）>

<一般,健壮>：是否进行无效数据的处理，一般只看有效等价类，健壮是有效+无效

单缺陷：一个测试用例仅覆盖**一个输入条件的某一个无效等价类**

弱一般等价类：单缺陷，正常值，测试用例总数= $\max\{m_1, m_2\}$

强一般等价类：多缺陷，正常值，测试用例总数= $m_1 * m_2$

弱健壮等价类：单缺陷，健壮值，测试用例总数= $\max\{m_1, m_2\} + \text{无效等价类个数}$

强健壮等价类：多缺陷，健壮值，测试用例总数= $(m_1 + l_1) * (m_2 + l_2) * \dots * (m_n + l_n)$

## • 例二: NextDate

### 输入定义域测试

| 输入条件的有效等价类           |
|----------------------|
| M1={月份 1<=月<=12}     |
| D1={日期 1<=日期<=31}    |
| Y1={年 1812<=年<=2012} |

| 输入条件的无效等价类    |
|---------------|
| M2={月份 月<1}   |
| M3={月份 月>12}  |
| D2={日期 日<1}   |
| D3={日期 日>31}  |
| Y2={年 年<1812} |
| Y3={年 年>2012} |

测试用例总数: (个)

| 弱一般 | 强一般 | 弱健壮 | 强健壮 |
|-----|-----|-----|-----|
| 1   | 1   | 7   | 27  |

弱一般: 单缺陷, 正常值, 实际上如果值正常了就不能说存在缺陷,  $\max\{1,1,1\}=1$

强一般: 强缺陷, 正常值,  $1*1*1=1$

弱健壮: 只是某个无效, 其他无效, 考虑无效值,  $\max\{1,1,1\}+\text{无效等价类个数}=7$

强健壮: 无效的全排列, 考虑无效值,  $(1+2) 3=27$

## • 例二: NextDate

弱健壮等价类测试用例

| month | day | year | 预期输出       |
|-------|-----|------|------------|
| 6     | 15  | 1912 | 1912年6月16日 |
| -1    | 15  | 1912 | 月份无效       |
| 13    | 15  | 1912 | 月份无效       |
| 6     | -1  | 1912 | 日期无效       |
| 6     | 32  | 2012 | 日期无效       |
| 6     | 15  | 1811 | 年无效        |
| 6     | 15  | 2013 | 年无效        |

测试用例总数: 7 个

强健壮等价类测试用例 (部分)

| month | day | year | 预期输出      |
|-------|-----|------|-----------|
| -1    | 15  | 1912 | 月份无效      |
| 6     | -1  | 1912 | 日期无效      |
| 6     | 15  | 1811 | 年无效       |
| -1    | -1  | 1912 | 月份、日期无效   |
| 6     | -1  | 1811 | 日期、年无效    |
| -1    | 15  | 1811 | 月份、年无效    |
| -1    | -1  | 1811 | 月份、日期和年无效 |

测试用例总数: 27 个

## 基于决策表的测试

等价类测试独立性假设, **忽略输入条件的相互关联 (适用于有关联的, 否否则效果和等价类一致)**, 所以测试用例存在严重的**冗余**, 决策表以等价类为基础, 是功能性测试方法中最严格的。

决策表的完备性保证一种完备的测试, 但不能保证无冗余

组成: 条件桩, 条件项, 动作桩, 动作项, 规则

实质: 针对个体输入域 有效等价类测试的扩充

- 1、**输入**条件都是**有效等价类**, 不是具体的数据
- 2、不考虑无效等价类
- 3、输出取值区不涉及输出域的等价划分

化简要求: 输出相同, 输入相似 (仅一个不同)

步骤：计算规则数、列出所有条件、动作、填入条件项和动作项，化简，合并规则

- 例：
- 问题要求：” ……对功率大于50马力的机器、维修记录不全或已运行10年以上的机器，应给予优先的维修处理……”
- 按5步建立判定表

- 计算规则数
- 列出所有条件、动作
- 填入条件项和动作项

|    |            | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|----|------------|---|---|---|---|---|---|---|---|
| 条件 | 功率大于50马力吗？ | Y | Y | Y | Y | N | N | N | N |
|    | 维修记录不全吗？   | Y | Y | N | N | Y | Y | N | N |
|    | 运行超过10年吗？  | Y | N | Y | N | Y | N | Y | N |
| 动作 | 进行优先处理     | x | x | X |   | X |   | X |   |
|    | 作其它处理      |   |   |   | X |   | x |   | x |

- 化简，合并规则

|    |            | 1 | 2 | 3 | 4 | 5 |
|----|------------|---|---|---|---|---|
| 条件 | 功率大于50马力吗？ | Y | Y | Y | N | N |
|    | 维修记录不全吗？   | Y | N | N | - | - |
|    | 运行超过10年吗？  | - | Y | N | Y | N |
| 动作 | 进行优先处理     | x | x |   | X |   |
|    | 作其它处理      |   |   | x |   | x |

## 基于正交表的测试

通过引入正交表，在数据完全**无规律**或测试人员**不了解数据规律**的条件下，仍可利用数据均布的特性来测试数据用例，避免测试方法的片面性，达到合理分布测试点并有效降低测试工作量的目的

试验点特性：**均匀分散，整齐可比**

常用的：L8 (2^7), L9 (3^4)

**Ln (q^s)**

n：实际测试用例的个数，对应正交表的行数

q: 每个输入条件所取测试数据的个数, 对应正交表中每个输入条件的取值个数(取值范围个数)  
s: 输入条件的总数, 对应正交表的列数  
qs: 理论上全组合方式的测试用例个数, 基于正交表的测试效率为n与qs的比值

性质:  
每一列中每个输入条件的各个测试数据出现的次数相同  
任意两列所构成的各有序数对出现的次数相同

## 基于场景的测试——基于用户行为的测试方法

对于某个功能点用户可能执行的操作有哪些, 测试就需要验证哪些  
一个场景测试用例仅测试一个场景、事务或业务流程。基于场景的测试主要集中在**用户和系统之间的交互**,主要用来检测业务需求的**正确性**,而不是代码本身的正确性。  
以**事件流**为核心, 是高层测试设计的基础  
结构: 基本事件流, 每个基本流代表一个被测的典型功能点或者主业务  
    备选事件流, 由触发事件产生的业务流程分支  
    场景, 由**基本流和备选流**可共同形成从开始到结束的不同**业务流程**  
    测试用例, 每个场景至少对应一组输入和一个预期输出结果  
基本流是从系统的某个**初始状态**开始, 经一系列状态变化后到达**终止状态**的过程中**最主要**的一个业务流程, 一个  
备选流是以**基本流**为基础, 在经过基本流上每个**判定节点**(包括条件判定和循环判定)处满足不同的触发条件, 而导致的其他事件流

|            | 基本流    | 备选流            |
|------------|--------|----------------|
| 测试重要性      | 重要     | 次要             |
| 数目         | 1 条    | 1 条或多条         |
| 初始节点位置     | 系统初始状态 | 基本流或其他备选流      |
| 终止节点位置     | 系统终止状态 | 基本流或系统其他终止状态   |
| 是否是完整的业务流程 | 是      | 否, 仅为业务流程的执行片段 |
| 能否构成场景     | 能      | 否, 需和基本流共同构成场景 |

基本流是一条从初始状态到终止状态的完成的业务流程, 而备选流仅仅是业务流程中的一个**执行片段**

### 场景设计的基本原则

- 1、最少的场景数等于**事件流的总数**, 即**基本流与备选流的总数**
- 2、**有且唯一有一个场景仅包含基本流**
- 3、对应某个备选流, **至少**应有一个**场景**覆盖该备选流, 且在该场景中应尽量**避免覆盖**其他的备选流

### 测试用例设计

- 1、**分析被测业务**, 基于风险的思想找到**基本流和所有备选流**;



- 2、根据基本流和备选流构造适当规模的场景；
- 3、根据场景设计测试用例；
- 4、对每个测试用例补充，并实施测试

简答🌴（个人猜测）通过分析被测**业务流程**，构建基本流和备选流，并生成场景进而得到测试用例的测试方法。该法主要用于**功能测试**

## 第4章 黑盒测试案例实践

### 1、保险金案例

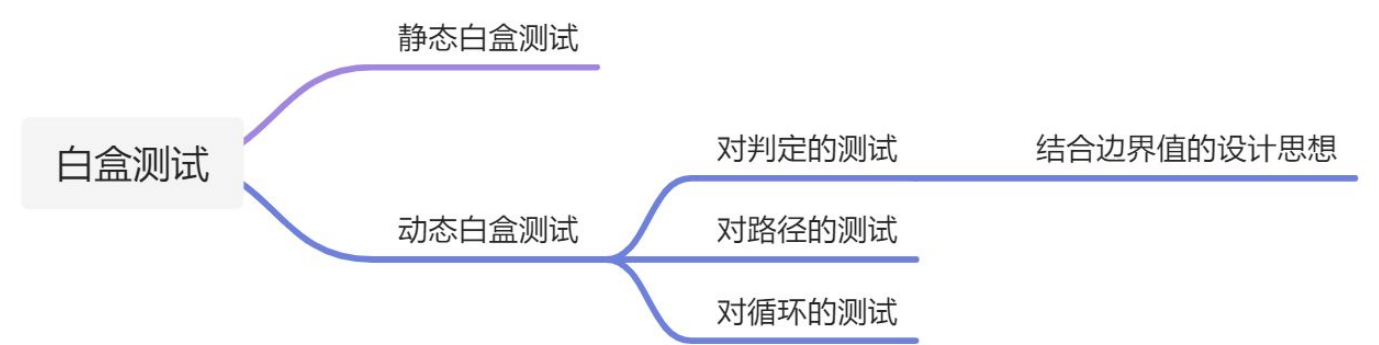
| 年龄 <sup>⓪</sup>           | 年龄系数 <sup>⓪</sup> | 门限分数 <sup>⓪</sup> | 安全驾驶折扣(单位：元) <sup>⓪</sup> |
|---------------------------|-------------------|-------------------|---------------------------|
| 16 ≤ 年龄 < 25 <sup>⓪</sup> | 2.8 <sup>⓪</sup>  | 11 <sup>⓪</sup>   | 50 <sup>⓪</sup>           |
| 25 ≤ 年龄 < 35 <sup>⓪</sup> | 1.8 <sup>⓪</sup>  | 9 <sup>⓪</sup>    | 100 <sup>⓪</sup>          |
| 35 ≤ 年龄 < 45 <sup>⓪</sup> | 1.0 <sup>⓪</sup>  | 7 <sup>⓪</sup>    | 150 <sup>⓪</sup>          |
| 45 ≤ 年龄 < 60 <sup>⓪</sup> | 0.8 <sup>⓪</sup>  | 5 <sup>⓪</sup>    | 200 <sup>⓪</sup>          |
| 60 ≤ 年龄 < 80 <sup>⓪</sup> | 1.5 <sup>⓪</sup>  | 7 <sup>⓪</sup>    | 250 <sup>⓪</sup>          |

包含的功能点很单一，不涉及业务流程，但包含复杂的输入/输出计算关系

### 2、信息采集系统案例实践

包含多个功能点，涉及业务流程，且包含用户界面来接受输入和提供处理结果的输出，需要进行单个功能点的测试和业务流程的测试

## 第5章 白盒测试技术



白盒测试也称**结构测试**或**逻辑驱动测试**，是针对被测单元内部是如何进行工作的测试。  
 白盒测试法检查程序**内部逻辑结构**，对**所有**逻辑路径进行测试，是一种**穷举路径**的测试方法。

即使每条路径都测试过了，仍然可能存在错误：

穷举路径测试无法检查出程序本身是否违反了设计规范，即程序是否是一个错误的程序。

穷举路径测试不可能查出程序因为遗漏路径而出错。

穷举路径测试发现不了与数据相关的错误。

原则：

保证一个模块中的所有独立路径至少被测试一次。

所有逻辑值均需测试真 (true) 和假 (false) 两种情况。

检查程序的内部数据结构，保证其结构的有效性。

在上下边界及可操作范围内运行所有循环。

白盒测试关注的对象：源代码，程序结构

当被测对象为函数时：源代码的逻辑是否符合该函数的功能要求，对应单元测试阶段，由开发人员执行

当被测对象为功能时：完成对业务流程的覆盖测试，对应集成测试甚至系统测试阶段，由测试人员执行

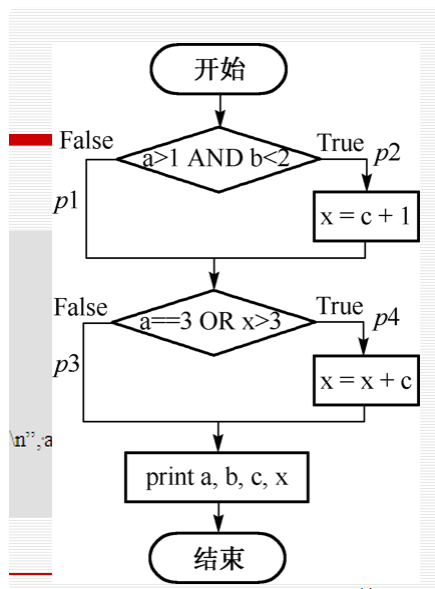
动态测试

侧重于关键程序结构的测试

基本思想：通过对导致程序结构复杂度的判定表达式、执行路径和循环结构设计测试用例，目标是达到某种程度的测试覆盖，从测试完备性和无冗余性的角度给予信心

典型方法：基于逻辑表达式覆盖指标的判定测试、基于全路径覆盖的独立路径测试，以及基于循环过程覆盖的对循环的测试

## 对判定的测试



语句覆盖：每一条可执行语句至少执行一次，对图中所有节点的覆盖 (24)

判定覆盖：每个判定节点的取真和取假分支至少执行一次 (24,13,14,23)

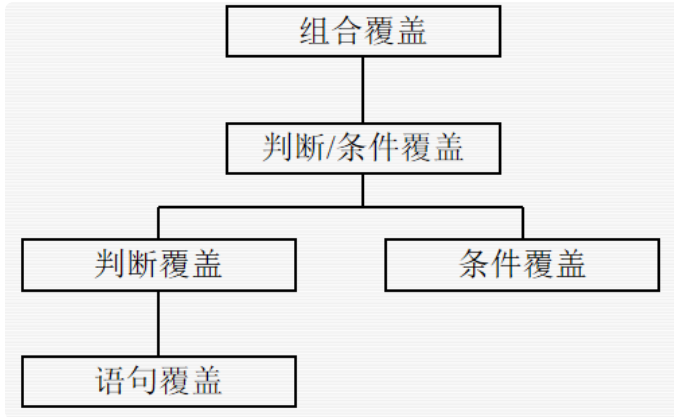
条件覆盖：每个复合判定表达式中，每个简单判定条件的取真和取假情况至少执行一次 (14,14)

条件覆盖不能保证100%的判定覆盖

判定/条件覆盖：判定覆盖+条件覆盖，判定节点，简单判定条件都取真取假

条件组合覆盖：每个判定节点中，所有简单判定条件的所有可能的取值组合情况应至少执行一次

修正的判定/条件覆盖：每个简单判定条件都应独立地影响到整个判定表达式的取值，利用独立影响性消除测试用例的冗余



## 对路径的测试

程序图：删注释，删变量声名，串为一，循环为一次

环复杂度：

直观观察法：程序图将二维平面分隔为封闭区域和开放区域的个数

公式计算法： $V(G) = e(\text{边}) - n(\text{节点}) + 1$

判定节点法： $V(G) = P + 1$  (P为判定节点的个数，有分支)

基本复杂度：无法再压缩后的环复杂度

测试用例设计：

对路径的测试中所需独立路径集合的大小就等于其程序图的环复杂度

独立路径抽取：1.确定主路径：包含尽可能多的判定节点，判定表达式，高的执行概率，多的语句

2、依据基础路径抽取其他独立路径

大题 🌱 测试用例设计步骤

根据程序源代码生成程序图

计算程序图的环复杂度，确定独立路径集合的大小

以最复杂的路径为基础路径，通过覆盖所有判定分支确定其他路径，抽取独立路径集合注意剔除不可行路径，必要时补充其他重要的路径

根据得到的路径集合对应设计测试用例

独立路径测试的理论基础保证了测试的完备性和无冗余性，基于程序图和环复杂度的独立路径测试**仅关注结构的测试覆盖**

## 对循环的测试

重点关注循环的过程正确性

分类：串联，嵌套，非结构化的循环

针对**单个循环节点**循环次数的测试：执行0,1, 2,  $n/2$ ,  $n-1$ ,  $n$ 次

针对多个循环**节点**循环过程的测试：初始化，迭代，终止

针对多个循环**结构**的测试：

串联

嵌套：内外，内内，外外，外内

非结构

对循环的测试重点在于观察循环过程是否符合设计，并不考虑循环体所涉及的相关变量所反映的结果有何实质含义，事实上，这也是所有白盒测试方法的局限所在

## 静态白盒测试

静态测试

侧重于**源代码检查和优化**

基本思想：**不需要设计测试用例，直接查看源代码和模拟执行代码**，目标是**直接定位代码中的缺陷**，提出结构设计优化的意见和有关测试重点的建议

**典型方法：同行评审、静态结构分析、代码质量度量和对变量的数据流测试**

### 1、代码检查

方法分类（正式——>随意）

**审查**、团队评审、走查、结对编程、同行桌查、轮差、**特别检查**

### 2、评审流程

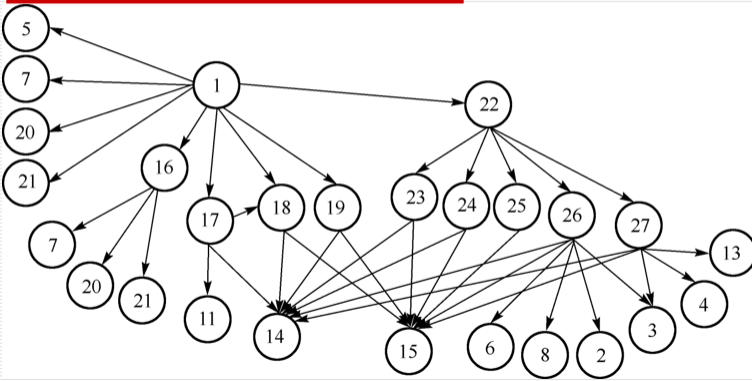
3、评审结果：正常，延期（超过30%未做好准备），取消

静态结构分析

基本原理：通过引入多种形式的**图表**(如函数调用关系图、模块控制流图等)，帮助人们快速了解**程序设计和结构**，更好地理解**源代码**，以及找到**程序设计缺陷**和代码优化的方向

函数调用关系图：函数之间的调用关系是否符合规定，是否存在递归调用，调用层次是否深，是否存在**孤立函数——>优先测试根节点，叶节点，接口数量多的节点**

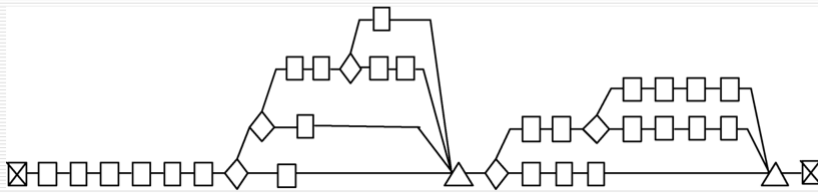
## □ 函数调用图



测试重点：1，（根节点）14,15，（入口多）26,27（出口多）

函数控制流程图:是否存在多出口，孤立语句，环复杂度大？非结构化？

## □ 函数控制流图



静态白盒测试是白盒测试的重要组成部分，它不需要执行程序，而是通过对比标准和规范，**检查程序逻辑，直接定位缺陷**

基于**缺陷预防**的思想，通过检查程序的各种图表定位那些具有高风险的程序代码，并承担部分代码质量度量的工作，可以更好地确保所提交的软件系统的质量

## 对变量的测试

以**被测变量**为中心，关注该变量的每条定义、使用路径，若该路径存在定义/引用异常缺陷，则该路径是一条高风险路径，需要重点进行测试

**定义/引用异常缺陷：1、变量在使用前未定义2、被定义未使用3、多次定义**

定义节点DEF(v,n)：被测变量的值在此处发生改变

使用节点USE(v,n)：被测变量在此处被使用

定义/使用路径du-path：从被测变量 的一个**定义节点开始**执行，到该变量的某个**使用节点结束**的一条路径

定义清除路径dc-path：某被测变量 的一条定义/使用路径中**不包含关于该变量的其他定义节点**

## 第6章 白盒测试案例实践

保险金案例

人寿保险金案例

信息采集系统

## 第7章 单元测试

单元测试(Unit Testing)是指对软件中的最小可测试单元或基本组成单元进行检查和验证

面向过程：单元是函数或者子过程

面向对象：单元是指类

静态测试：走查、审查，看代码，软件代码的模型

动态测试：对模块接口、模块边界条件、模块独立路径和错误处理进行测试

在被测模块的输入/输出域边界或其附近设计测试用例

驱动模块(Driver)是模拟被测单元的上级模块，用于接收测试数据、启动被测模块和输出结果

桩模块(Stub)是模拟被测单元所调用的模块。有时，需要使用子模块的接口，才能做少量数据操作，并验证和打印入口处的信息，然后返回。桩模块不包含原模块的所有细节

桩模块功能要求：在特定条件下完成原单元的基本功能，能够被正确调用。有返回值，不包含原单元的所有细节，只看接口参数和输出

Brian Marick：测试需求就是指明测试什么的规格说明

日构建(Daily Build)是自动、完整地构建整个代码库的代码，在构建的同时完成单元测试执行的一种软件研发工作模式

回归测试：贯穿在整个测试各个阶段的一个测试活动，主要是对修改过的软件重新进行测试，以保证验证修改的正确性及其影响

## 第8章 集成测试

集成测试就是在单元测试的基础上，将所有已通过单元测试的模块按照概要设计的要求组装为子系统或系统，并进行测试的过程，目的是确保各单元模块组合在一起后能够按既定意图协作运行，并确保增量的行为正确

必须经过单元测试后，才能够进行集成测试

单个集成测试用例的设计

- 成对集成：限定在一对调用单元上
- 邻居集成：限定在某个节点的邻居上，测试用例应同时包含该模块及其邻居
- 基于独立路径的集成：每条独立路径即可构成一个集成测试用例

集成测试遍历顺序的设计

- 大爆炸集成：将所有经过单元测试的模块一次性组装到被测系统中进行测试
- 自顶向下的集成(Top Down)：从主控模块(主程序，即根节点)开始
- 自底向上的集成(Bottom Up)：从底层模块(即叶子节点)开始
- 三明治集成(Sandwich)：分别自顶向下和自底向上展开集成，并在子树上进行大爆炸集成

| 项目   | 测试用例数目     | 桩模块 | 驱动模块 | 缺陷定位 | 并行测试 | 系统概貌 |
|------|------------|-----|------|------|------|------|
| 成对集成 | 由边数决定      | 需要  | 需要   | 非常容易 | 可以   | 不确定  |
| 邻居集成 | 主要由中间节点数决定 | 需要  | 需要   | 困难   | 可以   | 不确定  |
| 大爆炸  | 少          | 不需要 | 不需要  | 非常困难 | N/A  | 早期   |
| 自顶向下 | 较多         | 需要  | 不需要  | 较容易  | 困难   | 早期   |
| 自底向上 | 较多         | 不需要 | 需要   | 较容易  | 可以   | 较晚   |
| 三明治  | 较多         | 需要  | 需要   | 较困难  | 可以   | 早期   |

## 第9章 系统测试

系统测试是公司和项目组最关心的测试阶段

- 系统测试：将集成测试与计算机硬件、外部设备、支持软件、数据及人员等其他系统元素结合在一起，在实际使用(运行)环境下对计算机系统进行一系列的严格测试来发现软件中的潜在缺陷，保证系统交付给用户之后能够正常使用
- 系统测试不仅限于软件，不能省略

功能测试是系统测试中最基本的测试

以数据为中心的系统

- 核心是数据处理
- 从实体关系模型设计测试：1对1，m对n
- 从对数据的操作设计测试：增删改查

以活动序列为中心的系统

核心是活动序列

性能测试

- 常规性能测试：正常情况下
- 压力测试：持续不断地给被测系统增加压力，直至被测系统被压垮，以确定系统能承受的最大压力

负载测试：让被测系统在其能忍受的压力极限范围内(或**临界状态下**)连续运行，来测试系统的**稳定性**  
负载测试侧重于压力持续的时间，压力测试则更加强调施加压力的大小  
可靠性测试，大数据量测试

安全性测试

兼容性测试：检验被测软件与其他软、硬件相互是否能够正确交互和实现信息共享  
用户界面测试，可安装性测试